

FINAL PROJECT

Building and Evaluating a Feature-Based Grammar of English in DCG

Computational Linguistics

Pair project — 3 weeks — 100 points

Source text. Noel Burton-Roberts, *Analysing Sentences: An Introduction to English Syntax*, 5th edition, Routledge 2022 — Chapters 1-10. Only the constructions explicitly listed in this brief are assessable; students are not expected to implement every construction discussed in those chapters.

Environment. SWI-Prolog 9.0 or later. Standard library only (`plunit`, `lists`, `apply`). No external parsing toolkits.

Starter kit. `grammar_core.pl`, `harness.pl`, `tests_public.pl`. Distributed with this brief; verified against SWI-Prolog 9.0.4 (36/36 tests pass).

Team and schedule. Exactly two students per team; three milestones (Day 7, Day 15, Day 21). Milestones M1 and M2 are *gating*: a team that has not passed them is not admitted to the defence.

Contents

1 Rationale and learning outcomes	3
2 The task in one paragraph	3
3 The formal idiom (non-negotiable)	3
3.1 Feature records are compound terms	4
3.2 Every rule builds a phrase marker	4
3.3 Prohibited constructs	4
4 Part A — Baseline consolidation (Week 1)	5
4.1 What you are given	5
4.2 A1 — Get it running and understand it	5
4.3 A2 — Diagnose the seeded defects	5
4.4 A3 — Extend the lexicon	6
5 Part B — Extensions (Weeks 1-2)	6
5.1 B1 — The internal structure of NP (Chapter 7)	6
5.2 B2 — Sentences within sentences (Chapter 8)	6
5.3 B3 — <i>Wh</i> -clauses and relative clauses (Chapter 9)	7
5.4 B4 — Non-finite clauses (Chapter 10)	7
6 Part C — Evaluation and error analysis	7
6.1 C1 — Build the test suite	8

6.2 C2 — Measure	8
6.3 C3 — Analyse	9
7 Part D — The report and the defence	9
7.1 D1 — Report	9
7.2 D2 — Individual defence	9
7.3 D3 — Individual contribution evidence	10
8 Schedule and milestones	10
9 Deliverables and submission	10
10 Assessment	11
10.1 How the B components are marked	11
10.2 Penalties	12
11 Academic integrity and use of generative AI	12
A Idiom cheat sheet	12
B The harness	13
C Report template	13
D A worked extension	13

1 Rationale and learning outcomes

The module exercises (Modules 1–10) trained *technique*: each exercise isolated one phenomenon and gave you the rule schema to complete. This project asks for something categorically different — *grammar engineering under empirical evaluation*. You will be given a working but deliberately incomplete grammar, and you must extend it, break it, measure it, and argue about it in writing.

The central intellectual claim of the course is that a phrase-structure analysis is a *scientific hypothesis*: it predicts a set of strings, and it is refuted by any grammatical string it rejects and by any ungrammatical string it accepts. A grammar that has never been run against negative evidence has never been tested. Accordingly, half the credit in this project attaches not to code that works, but to evidence that you know precisely where and why it fails.

On completion you will be able to

- L01.** Encode the constituent structure of Chapters 1–10 in a feature-based DCG using a single, consistent formal idiom, and construct phrase markers as a by-product of parsing.
- L02.** Express selectional dependencies (subcategorisation, agreement, case, auxiliary form selection, gap threading) as *unification constraints* rather than as procedural tests.
- L03.** Design a test suite containing both positive and negative evidence, and quantify coverage, false acceptance on a negative challenge set, and structural ambiguity.
- L04.** Diagnose the three characteristic failure modes — **undergeneration**, **overgeneration**, **non-termination** — and trace each to a specific rule.
- L05.** Argue in writing for one syntactic analysis over an available alternative, using constituency evidence and observed parser behaviour.

2 The task in one paragraph

Starting from the supplied baseline grammar covering selected constructions from Chapters 2–6, extend it to cover the constructions of Chapters 7–10 — the internal structure of the noun phrase, subordinate clauses, *wh*-clauses and relative clauses, and non-finite clauses — keeping strictly to the formal idiom of the baseline. Build your own evaluation suite of at least 40 grammatical and 20 ungrammatical strings. Measure coverage, false acceptance and structural ambiguity. Diagnose every failure. Then write a detailed scientific report (15–20 pages) that presents the feature system formally, justifies your design decisions against the alternatives you rejected, reports the measurements, analyses the residual errors, and documents each member’s contribution. Each member defends the whole grammar individually.

This is a two-person project. Primary responsibility may be divided, but both members remain accountable for the complete grammar. Individual accountability is established through the contribution log, cross-review records, and a separate defence in which each member explains and modifies both their own component and their partner’s component. Section 11 states the policy on collaboration and generative AI.

3 The formal idiom (non-negotiable)

Every rule you write must conform to the idiom of the baseline. This is not a stylistic preference: the grading harness, the public tests and the reference solutions all assume

it, and a grammar written in a different idiom cannot be composed with the baseline you were given.

3.1 Feature records are compound terms

Features are carried in Prolog compound terms whose argument positions are fixed by the record's functor:

```
np_feat(Agr, Case)           Agr = agr(Person, Num)
vp_feat(Agr, Form)
lex_feat(Agr, Form, Subcat)
aux_feat(Agr, Form, Function, Selected)
pp_feat(PForm)
```

with $\text{Form} \in \{\text{fin}(\text{pres}), \text{fin}(\text{past}), \text{bare}, \text{en}, \text{ing}\}$ and $\text{Case} \in \{\text{nom}, \text{acc}\}$.

Read a record as an attribute-value matrix in which the attribute is the argument *position*. The rule

$$S \rightarrow \text{NP} \begin{bmatrix} \text{agr} \boxed{1} \\ \text{case } \textit{nom} \end{bmatrix} \text{VP} \begin{bmatrix} \text{agr} \boxed{1} \\ \text{form } \textit{fin} \end{bmatrix}$$

is written with a shared Prolog variable, and agreement is enforced by unification alone:

```
sentence(s(NP, VP)) -->
    np(NP, np_feat(Agr, nom)),
    vp(VP, vp_feat(Agr, fin(_))).
```

Feature lists such as `[cat:verb, subcat:trans|_]` are prohibited, as is any `val/3`-style accessor. Constraint satisfaction is pattern matching; if you find yourself writing a predicate that *looks up* a feature, you have left the idiom.

3.2 Every rule builds a phrase marker

The first argument of every non-terminal is its tree. A rule that recognises without constructing is incomplete, because Chapter 2's claim is precisely that a sentence *has* a structure, and you cannot evaluate a structural claim you never computed.

3.3 Prohibited constructs

- `assert/1`, `retract/1` — the grammar must be a pure, declarative object.
- Cuts inside grammar rules. (Cuts are permitted in the harness and in auxiliary arithmetic predicates only.)
- Left recursion of the form `vp --> vp, adjunct` or `nominal --> nominal, pp`. Post-modification must be handled by the accumulator technique already used in the baseline:

```
vp(VP, F) --> basic_vp(Base, F), post_adjuncts(Base, VP).

post_adjuncts(VP, VP) --> [].
post_adjuncts(Acc, VP) --> adjunct(Adj), post_adjuncts(vp(Acc, Adj), VP).
```

- Any transformation of the string before parsing (no pre-processing, no word re-ordering).

Why left recursion is fatal here, and why the accumulator is not merely a trick.

A DCG is executed by SWI-Prolog's top-down, depth-first resolution strategy. A rule whose leftmost daughter is its own mother re-enters itself with an unchanged goal and an unchanged input list; the derivation does not terminate, in *parsing* mode only if the string fails, but in *generation* mode always. The accumulator formulation is weakly equivalent — it licenses the same set of strings — but it makes the recursion right-

branching, so each recursive call consumes at least one terminal. You are required to state this argument, in your own words and applied to your own rules, in the report.

4 Part A — Baseline consolidation (Week 1)

4.1 What you are given

`grammar_core.pl` implements selected constructions from Chapters 2–6: the S rule with subject-verb agreement and nominative case on the subject; NP with `det` and the `nom` level with stacked adjunct APs; the six verb subcategories of Chapter 4 (`intrans`, `trans`, `ditrans`, `intens`, `complex`, `prep(PForm)`); VP-level adjuncts by accumulator; the auxiliary system `mod` > `perf` > `prog` > `pass`, where ordering is enforced by *form selection* rather than by an ordering list; and the passive as a parallel rule set rather than a `[voice]` feature.

`harness.pl` provides the measurement apparatus described in Section 6. `tests_public.pl` is the public acceptance suite: 36 `plunit` tests which your submission must still pass at the end of the project.

The baseline does *not* claim full coverage of Chapters 2–6. In particular, sentence adverbials, phrasal verbs, ellipsis, negation, *do*-support and subject-auxiliary inversion are outside the baseline unless a later task explicitly requires them. This distinction prevents lexical or constructional gaps from being mistaken for theoretical claims.

4.2 A1 — Get it running and understand it

```
$ swipl -g "run_tests, halt(0)" -g "halt(1)" tests_public.pl
% PL-Unit: baseline ..... passed
% All 36 tests passed
```

4.3 A2 — Diagnose the seeded defects

The baseline contains real limitations. The three documented cases below are the required diagnostic material. You are not required to hunt for undisclosed defects; credit is awarded for precise analysis rather than surprise discovery.

Known issue 1 — spurious ambiguity in the passive. The *by*-phrase is licensed twice: as the sister-of-V in `vp_passive`, and as a sister-of-VP adjunct through `post_adjuncts`. Verify:

```
?- n_pares([the,letter,was,written,by,kim], N).
N = 2.

?- n_pares([the,letter,has,been,written,by,kim], N).
N = 3.
```

Three parses for the second string, because the adjunct may now attach at the passive VP or at the perfect VP. *You must decide what the correct analysis is* — is the *by*-phrase a complement or an adjunct? — and justify the decision with constituency evidence from Chapter 5, not merely patch the symptom.

Known issue 2 — case on full NPs. Full NPs are given an unbound `Case`, so `the letter` is equally admissible in subject and object position, while pronouns are lexically specified. Is this an error or a correct statement about English morphology? Argue, do not assume.

Known issue 3 — lexical gaps masquerading as grammatical constraints. The baseline rejects `[kim, read, the, letter]` on its past-tense reading because the irregular past-tense entry is absent. This is a lexical-coverage failure, not a subcategorisation fact. By contrast, `[kim, considered, lucy]` is rejected because *consider* is encoded as complex-transitive and therefore selects an object-predicative. Distinguish these diagnoses in the report.

4.4 A3 — Extend the lexicon

Add 6–8 lexical items of your own, including at least two verbs, one noun that selects a PP, and one preposition. For each added verb, provide only the forms needed by your declared test suite and state explicitly which paradigm cells remain outside scope.

For reproducible grading, the following project vocabulary is mandatory and does not count toward the 15 items of your own: *say, ask, leave, arrive, want, persuade, writer, novel, beard, hair, all, of, because, that, whether, what, which, did, to* and the zero relativiser. Add every inflectional form actually required by the listed constructions. Hidden tests use this vocabulary only; they do not test unannounced words.

5 Part B — Extensions (Weeks 1-2)

Four extensions, all required. The rule fragments below are *targets*, not solutions: the shape of the rule is indicated, the feature threading is yours to work out.

5.1 B1 — The internal structure of NP (Chapter 7)

10 pts

Phenomena. The three-level structure NP – NOM – N; complement PPs (sister-of-N) versus adjunct PPs (sister-of-NOM); pre-determiners; and stacked post-modifying PPs.

Required coverage.

- `[a, writer, of, novels]` — PP complement, sister-of-N.
- `[the, student, with, long, hair]` — PP adjunct, sister-of-NOM.
- `[all, the, old, books]` — pre-determiner.
- Stacked post-modifiers, without left recursion.

Required diagnostic. For `[a, writer, of, novels, with, a, beard]`, draw the relevant phrase markers, identify the spans predicted to be replaceable by pro-NOM *one*, and encode at least two tests that distinguish the complement PP from the adjunct PP. A general substring-enumeration predicate is not required.

Trap. The obvious rule `nominal(nom(Nom,PP)) --> nominal(Nom), pp(PP,_)` is left-recursive. Use a `nominal_base` plus `nom_post_mods` accumulator, mirroring `basic_vp/post_adjuncts`. Note that the complement PP must attach *inside* the base and the adjunct PPs *outside* it — otherwise the structural distinction you have just drawn is invisible in the tree.

5.2 B2 — Sentences within sentences (Chapter 8)

12 pts

Phenomena. S-bar with an overt complementiser; *that*-clauses as direct objects; *whether* interrogative clauses; adverbial subordinate clauses; and lexical selection of the complementiser.

Required coverage.

- `[she, said, that, he, left]` — S-bar as dO.
- `[she, asked, whether, he, left]` — selected complementiser.

- [she, left, because, he, arrived] — adverbial clause as sister-of-VP.
- * [she, said, whether, he, left] must be *rejected*: verbs select their complementiser.
- * [she, asked, that, he, left] must be *rejected*.

Design question to answer in the report. A subordinate clause is a constituent of the matrix clause. Which node is it a sister of, and what constituency evidence decides the question? Your tree must reflect your answer.

5.3 B3 — *Wh*-clauses and relative clauses (Chapter 9)

14 pts

This is the technically hardest extension and the one that most rewards a clean feature design.

Phenomena. Object *wh*-questions with an NP gap; the gap as a threaded feature; and restrictive object relative clauses with *which*, *that* and zero relativisers. PP gaps, preposition stranding, pied-piping, subject gaps and island constraints are outside scope.

Required coverage.

- [what, did, she, write] — object gap.
- [the, book, which, she, wrote] — *wh*-relative.
- [the, book, that, she, wrote], [the, book, she, wrote].
- * [what, did, she, write, the, letter] must be rejected: the gap must be *filled exactly once*.

Mechanism. Thread a gap feature through the clause. A minimal signature:

```
% Gap = gap(np) | nogap
% s_with_gap(+Tree, +Gap) --> ...
% vp_gap(+Tree, +Features, +Gap) --> ...
```

The essential constraint is a *linear* one: the gap introduced at the top of the clause must be discharged at exactly one site. A grammar that permits zero discharges overgenerates * [what, did, she, write, the, letter]; one that permits two overgenerates worse. State in the report how your encoding guarantees uniqueness — and note that unification alone does not give you linearity for free.

5.4 B4 — Non-finite clauses (Chapter 10)

9 pts

Phenomena. *to*-infinitive clauses with overt and covert subjects, and the Type I / Type II contrast. *For*-clauses and *-ing* clauses are optional extensions, not required coverage.

Required coverage.

- [she, wants, to, leave] — covert subject.
- [she, wants, him, to, leave] — Type I (the NP is the subject of the subordinate clause).
- [she, persuaded, him, to, leave] — Type II (the NP is the direct object of the matrix verb, and the subordinate subject is covert).

The analytic point. Types I and II have identical surface strings, *V + NP + to-VP*, and different structures. Your grammar must assign them different phrase markers, and the difference must follow from a lexical feature on the verb, not from a rule that inspects the words. Chapter 10 gives the constituency diagnostics; cite the one you rely on.

6 Part C — Evaluation and error analysis

A grammar without a test suite is an untested hypothesis. This part carries 15 points and is where much of the intellectual work of the project actually happens.

6.1 C1 — Build the test suite

8 pts

Deliver `tests_own.pl` containing:

- **At least 40 grammatical strings**, distributed as follows: baseline 6, B1 8, B2 8, B3 10, and B4 8.
- **At least 20 ungrammatical strings**, distributed approximately as follows: baseline 3, B1 4, B2 4, B3 5, and B4 4; each is marked with the reason for ungrammaticality in a comment. Negative evidence is the scarce commodity: a suite of 200 positive items and no starred items tells you almost nothing, because a grammar that accepts every string in Σ^* passes it perfectly.
- **At least 8 minimal pairs** — a grammatical and an ungrammatical string differing in exactly one respect, so that the contrast isolates one constraint. `[she, has, written]` versus `*[she, has, writing]` isolates form selection; if your pair differs in two respects, it isolates nothing.
- **At least 4 ambiguity items** — strings you expect to receive more than one parse, with the expected count and a note on whether the ambiguity is genuine (two readings a speaker can perceive) or spurious (an artefact of your rules). This distinction is a graded item.

Draw your examples from the book where possible, and construct the rest. Cite the chapter and page for book examples.

6.2 C2 — Measure

3 pts

Run the supplied harness and report the metrics. With G the set of grammatical items and B the set of starred items, and $\text{acc}(\cdot)$ the subset the grammar accepts:

$$\text{Coverage} = \frac{|\text{acc}(G)|}{|G|}, \quad \text{ChallengeError} = \frac{|\text{acc}(B)|}{|B|}, \quad \bar{a} = \frac{1}{|\text{acc}(G)|} \sum_{s \in \text{acc}(G)} n(s)$$

where $n(s)$ is the number of distinct derivations of s .

```
?- evaluate(GoodList, BadList).

== EVALUATION =====
Coverage : 38/40 (0.95)
Challenge errors : 2/20 (0.10)
Mean ambiguity : 1.34 parses/sentence
-----
UNDERGENERATED (grammatical, rejected):
  [the,book,she,wrote,is,long]
  ...
=====
```

The second quantity is the false-acceptance rate on the submitted negative challenge set; it is not an estimate of overgeneration over all strings.

Also report a bounded generation check — $|\{s : |s| = n, s \in L(G)\}|$ for $n = 3, 4, 5$ — and the wall-clock time of the slowest item in your fixed test suite, measured over five runs on the same stated machine. A grammar whose generation counts explode combinatorially at $n = 5$ is telling you something about its ambiguity that the parsing numbers hide.

Perfect scores are permitted, but they are not sufficient evidence of quality.

Test-suite quality is assessed independently through phenomenon coverage, minimal contrasts and the diagnostic value of negative items. If no residual failure remains,

document at least three plausible boundary cases and explain why they are outside the declared grammar scope or how the grammar handles them.

6.3 C3 — Analyse

4 pts

For every residual failure, give: the string; the failure mode (undergeneration, false acceptance, non-termination, or spurious ambiguity); the specific rule responsible, identified by name; the minimal fix; and the reason you did or did not apply it. “I ran out of time” is an acceptable reason if the analysis is correct. A wrong diagnosis costs more than an unfixed bug.

On non-termination. If some query does not terminate, do not simply remove the rule. Characterise the class of inputs on which it diverges, identify the recursion that is not consuming input, and show the accumulator reformulation. A worked non-termination analysis is worth more than three fixed overgeneration bugs.

7 Part D — The report and the defence

7.1 D1 — Report

10 pts

Fifteen to twenty A4 pages, excluding cover page, table of contents, references, code appendix and full test appendix. Required structure:

1. **Abstract** (120 words). What you built, what it covers, what it fails on.
2. **Introduction and scope.** Which chapters, which constructions, what is deliberately out of scope.
3. **The feature system.** A formal presentation of your feature records as attribute-value matrices, and a statement of what unification does and does not enforce in your design. This section should be readable by someone who does not know Prolog.
4. **Implementation.** One subsection per extension B1–B4. Show the key rule, not the whole file. For each, name the alternative analysis you rejected and say why.
5. **Evaluation.** The metrics table, the minimal pairs, the ambiguity analysis.
6. **Error analysis.** Per Part C3.
7. **Limitations and future work.** Be specific. “The grammar does not handle coordination” is a limitation; “the grammar is not complete” is not.
8. **Contribution statement.** Identify the primary author and cross-reviewer for B1–B4, testing, integration and every report section.

For each B component, include the linguistic phenomenon, at least one phrase marker, the relevant feature structure, the key DCG rule, variable-sharing explanation, one minimal pair, one error analysis and one rejected alternative. Code in the report is illustrative only; do not paste more than 15 lines of Prolog in any one figure.

7.2 D2 — Individual defence

20 pts

Each member is examined separately for 10–12 minutes: explain one primarily owned rule; explain one partner-owned rule; predict an unseen test result; diagnose one failure; and make a small live modification. Both members are responsible for the entire grammar, and their final scores may differ.

7.3 D3 — Individual contribution evidence

5 pts

Submit a dated joint journal, a contribution matrix and version-history evidence. Each component must name a primary implementer and a cross-reviewer. Claims unsupported by concrete artefacts receive no credit.

8 Schedule and milestones

When	Work	Deliverable
Days 1-7	Run the starter kit; agree the feature contract and contribution plan; complete A, B1 and B2; begin the report.	M1 (gating). Public suite passes; B1-B2 work; at least 20 own tests; feature diagram; complement/adjunct design note.
Days 8-12	Implement the reduced B3; cross-review B1-B3; continue the report.	B3 demonstration: the NP gap is discharged exactly once, with minimal positive/negative evidence.
Days 13-15	Implement B4 and integrate the complete grammar.	M2 (gating). B1-B4 load together; public suite passes; integration tests and first metrics run.
Days 16-18	Complete 40 positive and 20 negative items; measurement and error analysis.	Metrics table, ambiguity inventory and residual-error dossier.
Days 19-21	Finalise the 15-20 page report; cross-check; code freeze and submission.	M3. Full submission. Individual defences follow.

The report must be written from Week 1. B3 typically takes two designs: an initial grammar that discharges gaps non-deterministically and a revised grammar that enforces a single discharge. A team that postpones B3 or the report until the final week is unlikely to complete either adequately.

9 Deliverables and submission

A single archive `<id1>_<id2>_final.zip` with exactly this layout:

```
<id1>_<id2>_final/
  grammar_core.pl % baseline plus your documented B1-B4 extensions
  tests_own.pl % your test suite (plunit)
  results.txt % raw output of evaluate/2 and the generation counts
  report.pdf % 15-20 pages, excluding specified front/back matter
  JOURNAL.md % dated joint work and AI-use log
  CONTRIBUTIONS.md % primary owner, cross-reviewer, evidence
  README.md % how to load and run, in 10 lines or fewer
```

Your submission must load and run cleanly with:

```
$ swipl -g "run_tests, halt(0)" -g "halt(1)" tests_own.pl
$ swipl -g "run_tests, halt(0)" -g "halt(1)" tests_public.pl
```

A submission that does not load is marked on the report alone, capped at 40. Warnings are acceptable; errors are not. Modifying `harness.pl` or `tests_public.pl` is not permit-

ted. You may and must extend `grammar_core.pl`; preserve its exported interface and mark every change with a comment of the form `% [B1]`, `% [B2]`, and so on. This single-module submission contract avoids implementation-dependent cross-module clause injection and makes the public commands above executable without a custom loader.

10 Assessment

	Component	Points
A	Baseline consolidation, lexicon extension, defect diagnosis	5
B1	NP internal structure and complement/adjunct analysis	10
B2	Subordinate clauses, complementiser selection	12
B3	<i>Wh</i> -questions, gap threading, relative clauses	14
B4	Non-finite clauses, Type I vs Type II	9
C1	Test suite: size, balance, minimal pairs, ambiguity items	8
C2	Measurement, correctly reported	3
C3	Error analysis: diagnosis quality	4
D1	Detailed team report	10
D2	Individual defence	20
D3	Individual contribution evidence	5
	Total	100

Components A–D1 contribute 75 team points. Components D2–D3 contribute 25 individual points. The two team members may therefore receive different final scores.

10.1 How the B components are marked

Each extension is marked on three axes, weighted roughly 2 : 2 : 1:

- **Coverage** — does it license the required strings and reject the required starred strings?
- **Structure** — is the phrase marker the right one? A grammar that accepts `[a, writer, of, novels]` with the PP as an adjunct scores at most half, because the string is right and the analysis is wrong.
- **Idiom** — does it conform to Section 3?

10.2 Penalties

Violation	Penalty
Feature lists or accessor predicates instead of feature records	−5 per rule set
Rule that recognises without building a tree	−3 each
assert/1 or retract/1 in the grammar	−10
Cut inside a grammar rule	−3 each
Grammar diverges on any item in the public suite	Part B capped at 50%
Public suite no longer passes	−10
Missing JOURNAL.md or CONTRIBUTIONS.md	−5 each

Penalties are capped at 20 points in total.

11 Academic integrity and use of generative AI

Collaboration within the registered two-person team is required. Sharing code, test suites or report text outside the registered team is not permitted. The contribution log must distinguish primary implementation from cross-review; division of labour does not remove either member's responsibility for the complete submission.

Generative AI may be used for explanation, debugging, and language editing, but not to generate the submitted analyses, test suite, or report. All AI use must be declared in JOURNAL.md. Undeclared or prohibited use will be handled under the institution's academic integrity regulations.

A Idiom cheat sheet

```
% --- Feature records -----
np_feat(Agr, Case) Agr = agr(Person, Num)
vp_feat(Agr, Form) Form = fin(pres) | fin(past)
lex_feat(Agr, Form, Subcat) | bare | en | ing
aux_feat(Agr, Form, Function, Selected)
pp_feat(PForm)

% --- Subcategories (Ch. 4) -----
intrans | trans | ditrans | intens | complex | prep(PForm)

% --- Agreement by shared variable -----
sentence(s(NP, VP)) -->
    np(NP, np_feat(Agr, nom)),
    vp(VP, vp_feat(Agr, fin(_))).

% --- Subcategorisation as unification -----
basic_vp(vp(V,D0), vp_feat(Agr,Form)) -->
    lexical_verb(V, lex_feat(Agr, Form, trans)),
    np(D0, np_feat(_, acc)).

% --- Adjuncts by accumulator (no left recursion) -----
vp(VP, F) --> basic_vp(Base, F), post_adjuncts(Base, VP).
post_adjuncts(VP, VP) --> [].
post_adjuncts(Acc, VP) --> adjunct(Adj), post_adjuncts(vp(Acc,Adj), VP).
```



```
% --- Auxiliary ordering by form selection -----
basic_vp(vp(Aux,VP), vp_feat(Agr,Form)) -->
    auxiliary(Aux, aux_feat(Agr, Form, Function, Selected)),
    { active_function(Function) },
    vp(VP, vp_feat(_, Selected)).

% --- Passive as a parallel rule set -----
basic_vp(vp(Aux,VPp), vp_feat(Agr,Form)) -->
    auxiliary(Aux, aux_feat(Agr, Form, pass, en)),
    vp_passive(VPp).
```

B The harness

```
parse(Words, Tree) % one parse, on backtracking all parses
accepts(Words) % succeeds at most once
rejects(Words)
parses(Words, Trees) % all derivations
n_parses(Words, N) % ambiguity count
generate(Len, Words) % bounded generation
sample(Len, Set) % distinct strings of that length
evaluate(Good, Bad) % prints the metrics block
show_tree(Words) % pretty-printed phrase markers
```

C Report template

```
Title, both names and student ids
Abstract (120 words)
1 Introduction and scope
2 The feature system
    2.1 Records as attribute-value matrices
    2.2 What unification enforces (and what it does not)
3 Implementation
    3.1 NP structure (B1)
    3.2 Subordination (B2)
    3.3 Wh- and relatives (B3)
    3.4 Non-finite clauses (B4)
4 Evaluation
    4.1 Test suite design
    4.2 Metrics
    4.3 Ambiguity: genuine vs spurious
5 Error analysis
6 Limitations and future work
7 Contribution statement
References
Appendices: full rule listing and full test suite
```

D A worked extension

The following is the *form* of an acceptable B1 answer for the complement/adjunct distinction — deliberately incomplete, to be finished by you. Note that the complement attaches inside `nominal_base` and the adjuncts outside it, so that the two attachment sites are distinguishable in the tree.


```
% NOM -> base (+ complement PP) then stacked adjunct PPs
nominal(Nom, Num) -->
    nominal_base(Base, Num),
    nom_post_mods(Base, Nom).

% sister-of-N: complement, selected by the noun
nominal_base(nom(N,PP), Num) -->
    noun_c(N, Num, PForm), % noun selecting a PP
    pp(PP, pp_feat(PForm)).
nominal_base(nom(N), Num) --> noun(N, Num).
nominal_base(nom(AP,Nom), Num) --> ap(AP), nominal_base(Nom, Num).

% sister-of-NOM: adjuncts, accumulated to the right
nom_post_mods(Nom, Num) --> [].
nom_post_mods(Acc, Nom) -->
    pp(PP, _),
    nom_post_mods(nom(Acc,PP), Nom).

noun_c(n(writer), sg, of) --> [writer].
```

Trace it by hand before you run it. For [a, writer, of, novels, with, a, beard], how many parses does this fragment give, and are they all the ones you want? The answer is not one, and understanding why is the exercise.